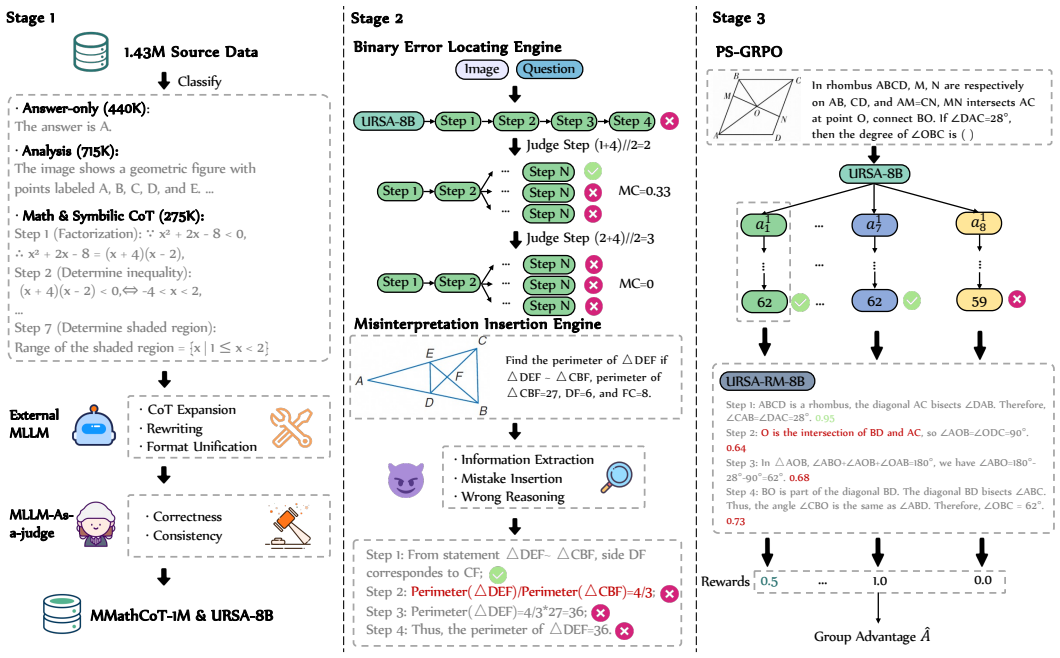

Figure 3: Pipeline of URSA. Stage 1 depicts the workflow of data curation as described in Section 2. Stage 2 illustrates how binary error locating and misinterpretation insertion facilitate the automation of process supervision data. Stage 3 demonstrates how our PS-GRPO operates by imposing penalties on rollouts that are questioned by the PRM.

We perform full-parameter instruction fine-tuning with MMathCoT-1M to train URSA-8B, based on the aligned model. The SFT dataset $\mathcal{D}_{SFT}$ is formed by the union of the curated solutions, i.e., $\mathcal{D}_{SFT} = \{(x_i, y_i) \mid (x_i, y_i) \in \mathcal{S}_{Ao} \cup \mathcal{S}_{An} \cup \mathcal{S}_C\}$. Training objective is demonstrated in Equation 1.

$$\mathcal{L}_{SFT} = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{SFT}} \sum_{t=1}^{T} \log \mathcal{M}(y_t|x, y_{<t}) \tag{1}$$

In this phase, we construct a stronger reasoning foundation model, URSA-8B, with the expectation of achieving a higher bound at inference time and to process supervision data of greater diversity.

3 Stage II: Dual-View Process Supervised Data Synthesis

3.1 Binary Error Locating Engine

Following suggestions by previous work [41–43], we train a PRM for first error step identification. We collect ∼553K incorrect solutions from URSA-8B's zero-shot inference on MMathCoT-1M. Erroneous steps in these solutions are labeled using Monte Carlo Tree Search (MCTS). For MCTS, an operation $\mathcal{F}(\{s_1, \ldots, s_i\}, N)$ generates N rollouts from a reasoning prefix $\{s_1, \ldots, s_i\}$. The single step's Monte Carlo estimation value, mc_i, is the fraction of these rollouts leading to a correct answer:

$$mc_i = \frac{|\text{Correct rollouts from } \mathcal{F}(\{s_1, s_2, \ldots, s_i\}, N)|}{|\text{Total rollouts from } \mathcal{F}(\{s_1, s_2, \ldots, s_i\}, N)|} \tag{2}$$

A step s_i is deemed "potentially correct" if $mc_i > 0$ [43, 42]. We optimize the identification of first error step using Binary Error Locating Engine (BEL): if the middle step has positive mc (i.e. $mc_{mid} > 0$), the error is in the latter half; otherwise, in the first (see Algorithm 1). To mitigate step-level label bias and include positive examples, we add ∼180K correct solutions (1/3 the number of incorrect ones), with all steps easily marked "True". This yields $\mathcal{S}_{BEL}$, a 773K process annotation dataset based on correctness potential.

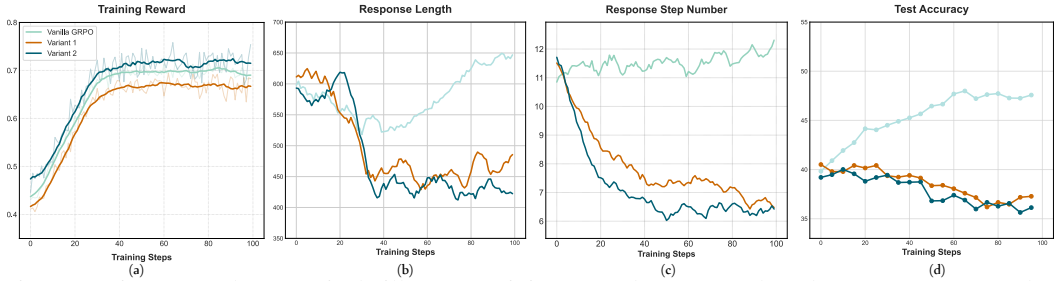

Figure 4: Figure (a)-(d) respectively illustrate training rewards, response length, response step number and test set accuracy of vanilla GRPO and two variants proposed in Section 4. Test set is randomly selected 500 examples from MMathCoT-1M for an in-domain evaluation.

3.2 Misinterpretation Insertion Engine

Apart from logical errors, the perception inconsistency between images and text in reasoning steps is a unique problem in multimodal scenarios [30, 44, 45]. We propose a Misinterpretation Insertion Engine (MIE) to artificially insert hallucinatory information, automatically constructing process supervision data with incorrect reasoning paths starting from the insertion point. Specifically, MIE includes three steps. First, we prompt $\mathcal{G}$ to perform a captioning task, extracting mathematical paradigm information from the image as much as possible. Second, the model $\mathcal{G}$ is required to focus on potentially confusable conditions within the existing correct solution and modify them using adjacent or similar conditions. Finally, the model $\mathcal{G}$ is prompted to continue reasoning based on the step with the inserted error. We leverage strong instruction-following capability of $\mathcal{G}$, instructing it to automatically assign negative labels to every subsequent step following the erroneous insertion. We generate $\sim$302K samples $\mathcal{S}_{MIE}$ using this strategy. Cases from MIE can be found in the Appendix H.2.

3.3 PRM Training

As shown in Equation 3, we merge two types of data, proposing a $\sim$1.1M process supervision data called DualMath-1.1M. During training, we append a special token after each step to indicate its predicted correctness. We model the PRM training as a binary classification task for the correctness of each step, as shown in Equation 4, here π_p is the trained PRM based on URSA-8B. e_j and y_j represent single step and corresponding label ($y_j \in \{0, 1\}$).

$$\mathcal{D}_{PRM} = \{(e, y_e) \sim \mathcal{S}_{BEL} \cup \mathcal{S}_{MIE}\} \tag{3}$$

$$\mathcal{L}_{PRM} = -\mathbb{E}_{(e,y)\sim\mathcal{D}_{PRM}} \sum_{j=1}^{|e|} \Big[y_j \log \pi_p(e_j) + (1 - y_j) \log(1 - \pi_p(e_j)) \Big] \tag{4}$$

Thus, Stage II delivers URSA-8B-RM, a strong PRM trained on DualMath-1.1M—the **first** large-scale, automatically labeled dataset for multimodal reasoning process supervision. While BoN evaluation demonstrates PRM's value in TTS, a critical question emerges: how can its guidance be directly integrated into MLLM post-training? This remains largely uncharted. Stage III draws a lesson about why previous scalar process reward modeling tends to fail, and then we achieve effective progress through process-as-outcome reward modeling.

4 Stage III: Integrating multimodal PRM into RL

Inspired by successes like DeepSeek-R1 [46], several recent studies have tried to adapt outcome reward-based GRPO for multimodal reasoning, demonstrating notable progress [47–50]. Outcome reward-based GRPO computes the i-th response's advantage through normalizing in-group rewards. However, outcome reward-based GRPO ignores the quality of reasoning processes [41, 51, 52].

Following most standard response-level and step-level reward modeling in RL [43, 28, 46, 13, 53], we examine two simple variants of GRPO with integrated scalar process rewards to reveal the failure patterns during the training process [54]. *Variant 1*: For i-th rollout, the reward is the sum of the

Figure 5: Figure (a) shows the BoN evaluation during GRPO training. We select the best rollout using the mean value of process rewards. Figure (b) illustrates the proportion of rollouts where URSA-8B-RM identifies "drop-moment" and the final results are indeed incorrect. Figures (c) and (d) display the response length and test accuracy during PS-GRPO training.

outcome reward and the average process reward, i.e. $r^i = r_o^i + \bar{r_s^i}$. *Variant 2*: Despite the outcome reward, a scalar process reward $r_{s,t}^i$ is assigned to the i-th rollout's t-th step. We observe two highly significant conclusions from Figure 4: **(i)** *High susceptibility to reward hacking*. The test accuracy of both variants is lower than vanilla GRPO. This indicates that when process scalar rewards are employed as learning objectives, the model quickly learns strategies that cater to process correctness. However, correctness in the process does not necessarily correlate fully with the heuristics leading to the ground-truth. **(ii)** *PRM's length bias in rewarding*. We observe a trend where increased training leads to shorter model responses and fewer reasoning steps. This phenomenon stems from an inherent length bias in the PRM's training labels; for examples with incorrect answers, steps taken after the first error are unlikely to yield a correct solution. This results in the PRM conservatively rewards the later stages of a reasoning rollout, thereby encouraging the MLLM towards more passive reasoning and a reliance on pattern recognition from existing conditions or simpler heuristics.

PS-GRPO The findings above confirm the consideration that flaws in the reward function are amplified when scalar process rewards serve as the optimization target [55, 33]. We ask "Which internal signals of PRM can be trusted?" We employ two views to investigate the reliable region of the PRM: first, the BoN performance during online learning, and second, the PRM's error identification capability. Regarding the latter, we introduce the concept of a *"drop-moment"* within the PRM's reward sequence, which signifies that the PRM questions the validity of the preceding steps. Specifically, for a given solution's PRM reward sequence $\{r_{p1}^i, r_{p2}^i, \cdots, r_{pN}^i\}$, a significant decrease in reward between consecutive steps indicates the occurrence of such a drop-moment.

$$\delta_p^i = \max\left\{ \frac{r_{p,j}^i - r_{p,j+1}^i}{r_{p,j}^i} \middle| j = 0, 1, \ldots, N-1 \right\} > \rho \tag{5}$$

Here, ρ represents PRM's drop-moment threshold. As illustrated in Figure 5, the PRM's ability for BoN selection and error identification remains largely unimpaired during the online RL process, exhibiting stable performance. This suggests that *although the scalar reward from the PRM in online RL might be unreliable, the relative quality of solutions it reveals is comparatively trustworthy*.

We leverage this beneficial property to address the reward sparsity problem in GRPO [56–58], aiming to make online RL focus more on learning from rollouts that have accurate results and rigorous processes. We use ρ from Equation 5 as the occurrence threshold for a "drop-moment"; when it occurs, we apply a reward penalty γ to rollouts with correct results. This both differentiates the learning value of outcome-correct rollouts and, due to its focus on relative drops in reward sequences, circumvents the impact of PRM's length bias in rewarding.

$$R^i = \begin{cases} 1, & o^i \text{ is correct and } \delta_p^i < \rho \\ 1-\gamma, & o^i \text{ is correct and } \delta_p^i \geq \rho \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

We utilize reward modeling in Equation 6 to conduct a process-supervised GRPO, which facilitates the computation of in-group advantages in Equation 7.